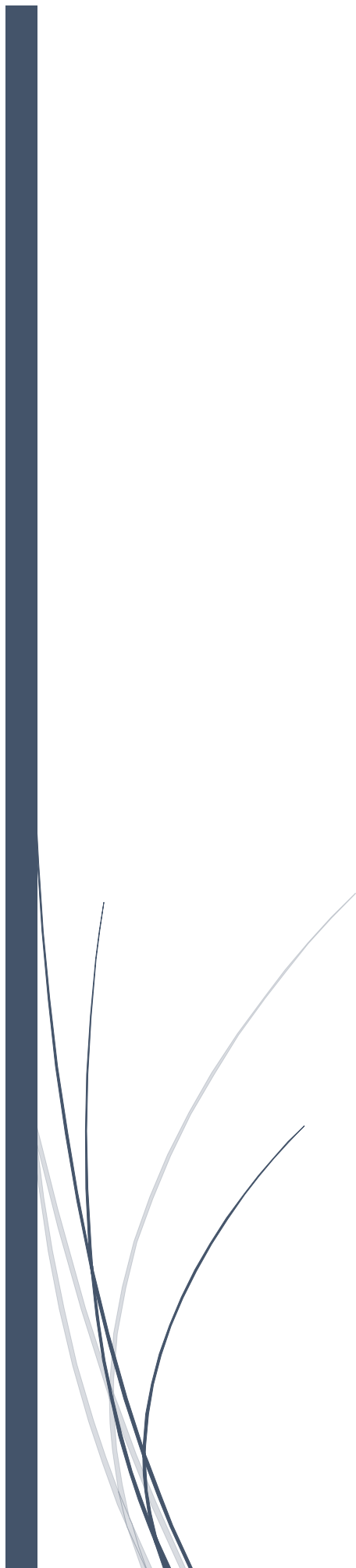


گزارش تمرین سوم DSD

طراحی CPU

اریسا ارزان پور - ۴۰۲۱۰۵۶۰۸

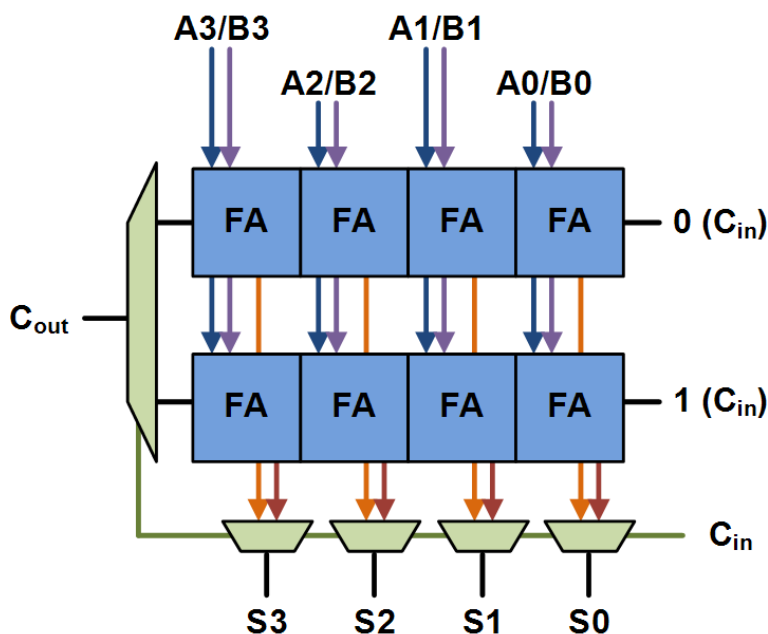


ماژول ها

۱- ماژول carry save adder (csa)

در این ماژول ما به ساخت جمع کننده به روش csa پرداختیم.

تصویر زیر مربوط به نمای شماتیک این جمع کننده است که در نوشتن کد آن استفاده شده است:



در ماژول csa، با برنامه نویسی سطح گیت به ساخت تصویر فوق پرداختیم.

سپس در ماژول csa_16bit ۴ بار از ماژول csa اینستنس گرفتیم و cout های هر بخش را به بخش بعدی

دادیم تا فرایند انتقال کری اتفاق بیفتد.

هرکدام از این ۴ اینستنس، ۴ بیت از جواب را به ما میدهند و همچنین اینستنس آخر Cout جمع کلی را نیز به ما میدهد.

همچنین ماژول Mux نوشته شده که منطق ساده ی مالتی پلکسر را پیاده سازی میکند.

۲- ماثول multiply با روش Karatsuba

طبق توضیحات داده شده در داک تمرین این بخش پیاده سازی شده است.

ابتدا به توضیح ماثول shiftadd میپردازیم.

این ماثول دو عدد ۸ بیتی را ورودی میگیرد و یک خروجی ۱۶ بیتی که حاوی حاصلضرب دو عدد ورودی است را خروجی میدهد. همچنین یک سیگنال done نیز را پس از انجام عملیات و اتمام آن برابر ۱ میکند و خروجی میدهد که بعداً در الگوریتم مورد استفاده قرار میگیرد.

از یک fsm با ۴ استیت برای انجام این روش استفاده میکنیم.

ابتدا چک میکنیم که آیا سیگنال reset یک است یا خیر، و در صورت یک بودن، تمام سیگنالها را ریست کرده و مقدار اولیه ۰ را به آنها میدهیم.

سپس در مرحله بعد در اولین استیت fsm اگر سیگنال start که ورودی ماثول بود (برای تعیین زمان شروع عملیات ضرب) یک بود به استیت بعدی میرویم.

در استیت ۰۱، ما به آماده سازی عوامل ضرب میپردازیم:

```
2'b01: begin
    result    <= 16'b0;
    multiplicand <= {8'b0, A};
    multiplier <= B;
    count     <= 3'd7;
    state     <= 2'b10;
end
```

برای multiplicand، ورودی A را با extend کردن ۱۶ بیتی میکنیم تا در شیفت مرحله بعد مورد استفاده قرار گیرد.

حال به استیت ۱۰ میرویم و در این استیت اگر کم ارزش ترین بیت multiplier یک باشد عددی که multiplicand دارد را با result جمع میکنیم و در result میریزیم. (مقدار اولیه result صفر است)

در ادامه ، چه آن بیت ۱ باشد و چه صفر ، ابتدا multiplicand را یک بیت به چپ شیفت میدهیم و multiplier را یک بیت به راست شیفت میدهیم تا رقم بعدی آن را مورد بررسی قرار دهیم.

این دقیقا معادل ضربی است که با قلم و کاغذ انجام میدهیم ولی چون ارقام ۰ و ۱ اند نیازی به ضرب داخلی ای نیست. در این مرحله چک میکنیم که آیا ۸ بار ساینکل این مراحل را انجام دادیم یا خیر. اگر انجام شده بود به استیت آخر که استیت فاینال fsm هم هست میرویم و حاصل نهایی را در متغیر مربوطه میریزیم و همچنین سیگنال done را یک میکنیم و دوباره به استیت ۰۰ برمیگردیم.

حال به ماژول اصلی multiply میپردازیم. در این ماژول ورودی های علامت دار را دریافت کرده و به تشخیص علامت آنها با توجه به MSB شان میپردازیم. سپس با استفاده از xor تشخیص میدهیم که نتیجه ضرب چه علامتی خواهد داشت تا نهایتا آن را روی نتیج اعمال کنیم. پس از تشخیص علامت ورودی ها در صورت منفی بودن ، مکمل دوی آنها را حساب میکنیم تا ضرب را با اعداد مثبت انجام دهیم.

با توجه به فرمول های گفته شده در داک تمرین ، اعداد را بخش بندی و با سه بار اینستنس گرفتن از ماژول shiftadd ، ضرب های گفته شده را محاسبه میکنیم.

سپس برای این ماژول نیز یک fsm تعریف میکنیم.

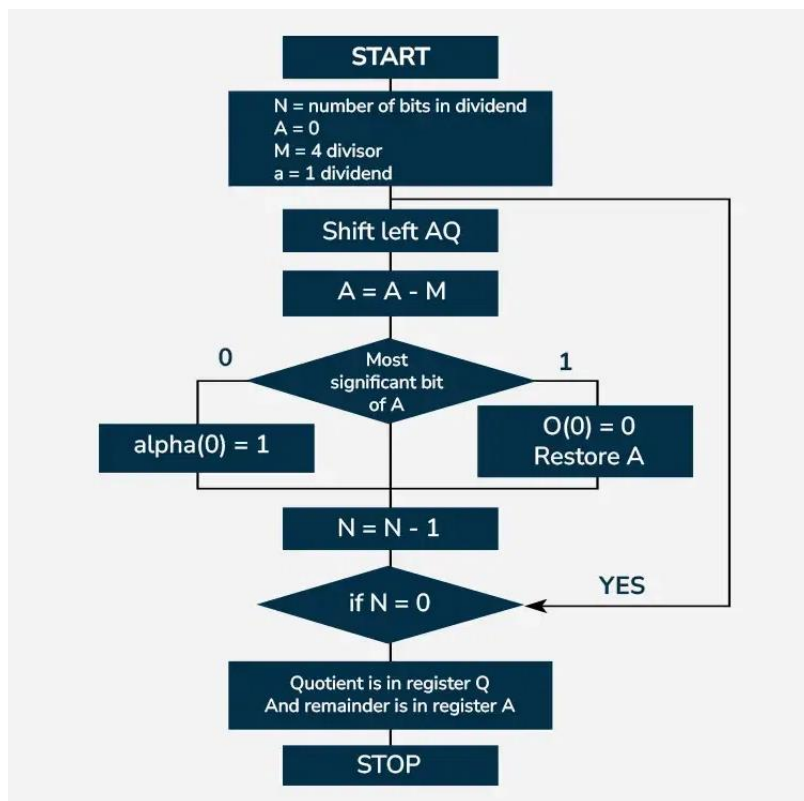
```
parameter WAIT = 2'b00, START_MULT = 2'b01, CALCULATE = 2'b10, DONE = 2'b11;
```

ابتدا مانند ماژول قبلی ، سیگنال ریست را چک میکنیم. در صورت فعال نبودن ، به اولین استیت ماشین حالت خواهیم رفت. در این مرحله در صورت دریافت سیگنال استارت که ورودی این ماژول است و به معنای آغاز فرایند است ، سیگنال های استارت آن سه اینستنس را یک کرده و به استیت بعدی میرویم. در استیت بعدی پس از فعال سازی آن استارت ها لازم است غیر فعالشان کنیم و سپس به استیت بعدی برویم.

در استیت بعدی که مربوط به محاسبه است ، محاسبه مقدار نهایی و حاصل ضرب کلی با توجه به فرمول های داک صورت میگیرد ، در صورتی که سیگنال پایان هر سه اینستنس که وظیفه انجام ضرب های ۸ بیتی را داشتند ، فعال شده باشد تا ما حاصل ضرب های کامل و صحیح را در اختیار داشته باشیم و استفاده کنیم. در همین استیت نیز بر اساس علامت ضرب که در مراحل قبل تعیین شده بود ، جواب نهایی را در متغیر مربوطه قرار میدهیم و نهایتا به استیت پایانی done میرویم و در آن سیگنال done که خروجی ای از این ماژول هست را یک میکنیم که در سایر ماژول ها بدانیم عملیات پایان یافته و مجددا به استیت اول میرویم.

۳- مازول division با روش Restoring

این مازول به لحاظ کلی ساختاری مانند مازول قبلی دارد با این تفاوت که مطابق الگوریتم زیر انجام شده است:



نام متغیر ها را نیز دقیقاً مطابق همین فلوجارت قرار دادیم. نهایتاً ، این مازول نیز یک سیگنال done دارد که در استیت پایانی fsm اش برابر ۱ خواهد شد تا پایان عملیات در مازول های بالاتر مورد استفاده قرار گیرد.

۴- مازول ALU

ماژول ALU در بر گیرنده تمامی مازول های فوق است. این مازول با دریافت سیگنال کلاک ، ریست ، دو عدد ۱۶ بیتی علامت دار و opcode ، این مقادیر و دیتا را به اینستنس های گرفته شده از مازول های فوق میدهد و نتیجه را براساس opcode دریافتی خروجی میدهد. همچنین یک سیگنال done نیز خروجی میدهد که بیانگر اتمام کار ALU است.

در بخش fsm این مازول ، سه استیت IDLE , EXECUTE , DONE دارد. در استیت IDLE ، ابتدا سیگنال های کنترلی ۰ میشوند و در صورتی که سیگنال start یک باشد ، براساس opcode و با یک case ، دستور R-type مدنظر تشخیص داده میشود و سیگنال های کنترلی آنها و ورودی مازول های مربوطه شان مطابق کد تعیین

میگردد. اگر opcode ورودی جز opcode های valid نبود ، در خروجی ALU ، عبارت dead را خواهیم داشت.

در نهایت همه ی کیس های صحیح در این استیت به استیت بعدی یعنی EXECUTE میروند.

```
EXECUTE: begin
case (opcode)
  3'b000, 3'b001: begin
    // for add and sub we take the result immediately (no sequential element, all gates)
    result <= add_result;
    done <= 1;
    state <= DONE_STATE;
  end
  3'b010: begin
    // for Mul and Div we need to wait until we receive the done signal from their modules, then we
    can proceed to DONE state and set the done signal.
    if (mul_done) begin
      result <= mul_out[15:0];
      done <= 1;
      state <= DONE_STATE;
    end
  end
  3'b011: begin
    if (div_done) begin
      result <= div_quotient;
      done <= 1;
      state <= DONE_STATE;
    end
  end
endcase
end
```

پس از این استیت نیز به استیت نهایی میرویم و در آن استیت صرفا به استیت اولیه IDLE بازمیگردیم.

۵- ماژول Memory

کد زیر مربوط به ماژول memory است. توضیحات مهم کامنت شده اند.

```
module memory(  
    input clk,  
    input write_enable,  
    input [15:0] address,  
    input signed [15:0] write_data,  
    output signed [15:0] read_data  
);  
  
    reg signed [15:0] mem [0:2047]; // making the size of the mem smaller so the  
    synthesis is possible  
  
    always @(posedge clk ) begin  
        if (write_enable) begin // Sync write when write_enable = 1  
            mem[address] <= write_data;  
        end  
    end  
  
    assign read_data = mem[address]; //Async read  
endmodule
```

۶- مازول Register File

کد زیر مربوط به مازول Register file است. توضیحات مهم کامنت شده اند.

```
module register_file (  
    input wire clk,  
    input wire reset      ,  
    input wire write_enable,  
    input wire [1:0] read_addr1,  
    input wire [1:0] read_addr2,  
    input wire [1:0] write_addr,  
    input wire [15:0] write_data,  
    output reg [15:0] read_data1,  
    output reg [15:0] read_data2  
);  
  
    reg [15:0] X[0:3];    //used registers  
  
    always @(negedge clk) begin    // on negedge of the clock the reading procedure is done.  
        read_data1 <= X[read_addr1];  
        read_data2 <= X[read_addr2];  
    end  
  
    always @(posedge clk or posedge reset) begin  
        //on posedge of the clock, writing procedure will be done. Also if reset = 1, all regs = 0  
        if (reset) begin  
            X[0] <= 16'b0;  
            X[1] <= 16'b0;  
            X[2] <= 16'b0;  
            X[3] <= 16'b0;  
        end else if (write_enable) begin  
            X[write_addr] <= write_data;  
        end  
    end  
end  
endmodule
```

۷- مازول Control Unit

حال به بررسی مهم ترین مازول میپردازیم.

در این مازول ما خروجی مازول هایی که تا اینجا بررسی شده اند را به عنوان ورودی داریم و همچنین ورودی آنها نیز ، خروجی مازول کنترل خواهد بود تا بتوان منطق پشت استفاده از آنها را پیاده سازی کرد.

همچنین یک سیگنال خروجی ready داریم که هرگاه یک دستور انجام شد ، برابر یک میشود.

ساختار اصلی این مازول یک fsm پنج حالتی مانند میپس است. شامل WB , MEM , EXE , ID , IF .

در استیت IF ، ابتدا PC را به عنوان آدرس مموری میدهیم تا بتوانیم دستور را در مراحل بعدی از مموری استخراج و اجرا کنیم (PC یک متغیر رجیستر درونی است و در بخش ریست مقدار ۰ در آن قرار داده میشود و پس از آن در آخرین استیت به آن یکی اضافه میشود و جلو میرود) و همچنین بقیه سیگنال های صفر میمانند. پس از آن به استیت ID میرویم. در این مرحله دیتا خوانده شده از مموری را در یک رجیستر بنام instruction میریزیم تا آن را دیکود کنیم. ابتدا Opcode استخراج میشود و بر اساس آن ، بسته به اینکه دستور R-type باشد یا M-type سایر بیت های آن نیز مشخص میشوند. پس از این مرحله ، استیت EXE اجرا میشود.

در این استیت ، برای opcode های R-type ورودی های ALU ، به ALU داده میشوند که مقادیر عددی داده شده به ALU در واقع خروجی های Regfile هستند که آدرس رجیستر ها از طریق read_address به آن داده شده است و خروجی مقادیر موجود در رجیستر هاست که در ALU استفاده میکنیم. پس از آن سیگنال شروع ALU را یک میکنیم و زمانی که سیگنال done ALU برابر یک شد ، استارتش را صفر کرده و به WB میرویم چون این دستورات نیازی به MEM ندارند.

برای دستورات M-type نیز مطابق R-type ها ، ورودی های ALU را مطابق چیزی که نیاز دارند میدهیم و وقتی سیگنال done برابر ۱ شد ، خروجی ALU را به عنوان آدرسی که قرار است به مموری بدهیم قرار میدهیم (که در آن بنویسیم و یا بخوانیم). بقیه بخش های آن مطابق توضیحات R-type هاست. این دستورات سپس به استیت MEM میروند.

در استیت MEM بسته به اینکه دستور Load است یا Store ، سیگنال های کنترلی مموری فعال میشوند . برای دستور Load پس از این مرحله به WB میرویم اما برای Store به IF باز میگردیم و همچنین PC را آپدیت میکنیم.

حال در استیت آخر یا WB سیگنال های کنترلی Regfile را برای نوشتن فعال میکنیم. همچنین برای Load خروجی مموری را ذخیره میکنیم و برای R-type ها خروجی ALU را.

۸- مازول CPU_top

این مازول دو سیگنال کلاک و ریست را به عنوان ورودی دارد. در این مازول منطقی پیاده نمیشود و صرفاً تمامی سیگنال های استفاده شده در مازول های پیشین را به عنوان wire در آن تعریف میکنیم و از مازول ها اینستنس میگیریم تا اتصالات میان آنها را ایجاد کنیم.

خروجی تست بنچ ها:

۱- مقادیر ذخیره شده در حافظه و رجیستر ها و دستورات در تست بنچ اول:

```
uut.memory_inst.mem[0] = 16'h8205; // LOAD R0, R1, #5
uut.memory_inst.mem[1] = 16'h0300; // ADD R0, R1, R2
uut.memory_inst.mem[2] = 16'hA206; // STORE R0, R1, #6
uut.memory_inst.mem[3] = 16'h3B00; // SUB R3, R1, R2
uut.memory_inst.mem[4] = 16'h5080; // MUL R2, R0, R1
uut.memory_inst.mem[5] = 16'h6D80; // DIV R1, R2, R3

uut.regfile.X[0] = 16'd0;
uut.regfile.X[1] = 16'd10;
uut.regfile.X[2] = 16'd20;

uut.memory_inst.mem[15] = 16'd123;
```

خروجی مشاهده شده:

```
VSIM 190> run -all
# Time: 0 | R0: 0 | R1: 0 | R2: 0 | R3: 0 | Mem[16]: x
# Time: 20 | R0: 0 | R1: 10 | R2: 20 | R3: 0 | Mem[16]: x
# Time: 105 | R0: 123 | R1: 10 | R2: 20 | R3: 0 | Mem[16]: x
# Time: 175 | R0: 30 | R1: 10 | R2: 20 | R3: 0 | Mem[16]: x
# Time: 245 | R0: 30 | R1: 10 | R2: 20 | R3: 0 | Mem[16]: 30
# Time: 315 | R0: 30 | R1: 10 | R2: 20 | R3: -10 | Mem[16]: 30
# Time: 525 | R0: 30 | R1: 10 | R2: 300 | R3: -10 | Mem[16]: 30
# Time: 775 | R0: 30 | R1: -30 | R2: 300 | R3: -10 | Mem[16]: 30
#
# --- Final Register Values ---
# R0 = 30
# R1 = -30
# R2 = 300
# R3 = -10
# Mem[16] = 30
# ** Note: $stop : C:/Users/AA_NOIR/Desktop/project_dsd/tb_1.v(60)
```

۲- مقادیر ذخیره شده در حافظه و رجیستر ها و دستورات در تست پنج دوم:

```
uut.memory_inst.mem[0] = 16'h2300; // SUB R0, R1, R2 => R0 = R1 - R2 = -17 - (-5) = -12
uut.memory_inst.mem[1] = 16'h5900; // MUL R3, R0, R2 => R3 = -12 * -5 = 60
uut.memory_inst.mem[2] = 16'h7680; // DIV R2, R3, R1 => R2 = 60 / -17 = -3 (integer div)
uut.memory_inst.mem[3] = 16'h0900; // ADD R1, R0, R2 => R1 = -12 + (-3) = -15
uut.memory_inst.mem[4] = 16'hB7FF; // STORE R2, R3, #-1

uut.regfile.X[0] = 16'd0;
uut.regfile.X[1] = -17;
uut.regfile.X[2] = -5;
uut.regfile.X[3] = 0;
```

خروجی مشاهده شده:

```
-----
# Time: 0 | R0:      0 | R1:      0 | R2:      0 | R3:      0 | Mem[59]:      x
# Time: 20 | R0:      0 | R1:    -17 | R2:     -5 | R3:      0 | Mem[59]:      x
# Time: 95 | R0:    -12 | R1:    -17 | R2:     -5 | R3:      0 | Mem[59]:      x
# Time: 305 | R0:    -12 | R1:    -17 | R2:     -5 | R3:     60 | Mem[59]:      x
# Time: 555 | R0:    -12 | R1:    -17 | R2:     -3 | R3:     60 | Mem[59]:      x
# Time: 625 | R0:    -12 | R1:    -15 | R2:     -3 | R3:     60 | Mem[59]:      x
# Time: 695 | R0:    -12 | R1:    -15 | R2:     -3 | R3:     60 | Mem[59]:     -3
#
# --- Final Register Values ---
# R0      =    -12
# R1      =    -15
# R2      =     -3
# R3      =     60
# Mem[59] =     -3
```